

Rapport Défi Développement Durable  
Spécialité Électronique et Télécommunications



# Station télémétrique pour avion RC

FIMI 2A – ENSIL-ENSCI – 2025-2026

Clovis Laforêt  
Clément Letranchant



# Sommaire

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Sommaire.....</b>                                              | <b>2</b>  |
| <b>Introduction.....</b>                                          | <b>3</b>  |
| <b>Architecture matérielle.....</b>                               | <b>4</b>  |
| Unités de contrôle : Arduino Nano Every et Arduino Mega 2560..... | 4         |
| Unité de capture vidéo : ESP32-S3 IA CAM.....                     | 4         |
| Capteur de température, d'humidité et de pression : BME280.....   | 5         |
| Module GPS : U-blox NEO-7M.....                                   | 5         |
| Communication longue portée : LoRa SX1278.....                    | 6         |
| Énergie.....                                                      | 7         |
| Dimensionnement énergétique.....                                  | 7         |
| Autonomie.....                                                    | 7         |
| <b>Station sol.....</b>                                           | <b>8</b>  |
| Retour des coordonnées GPS.....                                   | 8         |
| Retour des informations télémétriques.....                        | 8         |
| Exploitation vidéo post-vol.....                                  | 9         |
| <b>Câblage et Programmation.....</b>                              | <b>10</b> |
| LED d'état.....                                                   | 11        |
| Distribution de l'énergie.....                                    | 11        |
| Programmation.....                                                | 11        |
| <b>Informations complémentaires.....</b>                          | <b>12</b> |
| Sécurité et contraintes.....                                      | 12        |
| Fréquences.....                                                   | 12        |
| Communication LoRa coupée.....                                    | 12        |
| Améliorations envisageables.....                                  | 12        |
| <b>Conclusion.....</b>                                            | <b>13</b> |
| <b>Bibliographie.....</b>                                         | <b>14</b> |
| <b>Annexes.....</b>                                               | <b>15</b> |



## Introduction

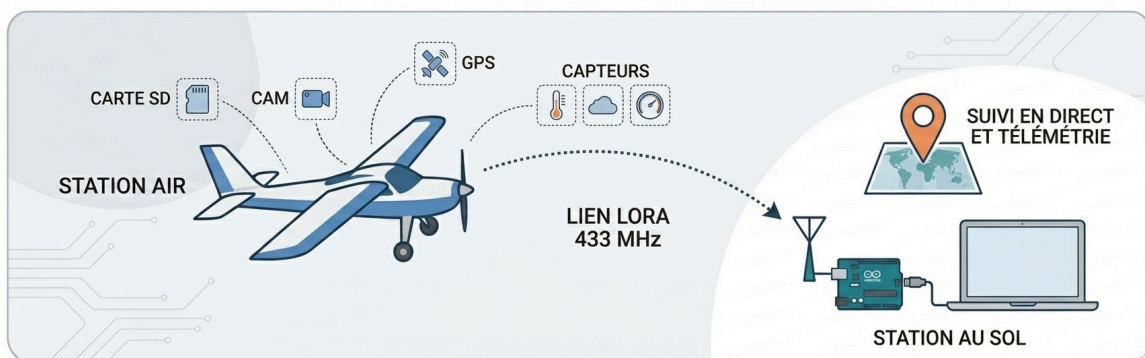
Étant en possession d'un avion RC télécommandé dépourvu de capteurs de vol, nous nous sommes intéressés à la possibilité de créer par nous-même cette "station télémétrique".

En effet, l'objectif du projet est d'ajouter à un modèle RC un groupe de capteurs, dont les données récoltées sont transmises au télépilote au sol. Nous avons déterminé que les informations les plus importantes et intéressantes à apporter à cet avion sont la température, la pression, l'humidité, l'altitude, et la position GPS. Elles permettent d'avoir un suivi qualitatif quant au déroulement du vol. De plus, il est intéressant pour un télépilote d'avoir un retour vidéo, c'est pourquoi une caméra embarquée complète le projet.

En permettant la collecte de données atmosphériques ainsi que le suivi géographique précis d'un aéronef léger, cette station télémétrique constitue un outil d'analyse pouvant être utilisé pour étudier des phénomènes locaux. Ce type de dispositif permettrait de réaliser des relevés à moindre coût et avec un impact environnemental limité, notamment en comparaison avec des moyens d'observation plus lourds et énergivores.

Ainsi, à travers ce rapport, vous découvrirez notre cheminement de pensée, les obstacles rencontrés, et les solutions apportées, afin d'avoir un retour complet sur ce que nous avons imaginé pour atteindre notre objectif :

**Concevoir une solution autonome capable de capturer des données environnementales, de transmettre sa position géographique en temps réel à une station au sol via une liaison longue portée, et de filmer et enregistrer des images pour une exploitation vidéo post-vol.**



## Architecture matérielle

### Unités de contrôle : Arduino Nano Every et Arduino Mega 2560

Le système repose sur une Arduino Nano Every et une Arduino Mega 2560. Ces cartes ont été retenues pour leur importante documentation, leur facilité d'intégration et leur nombre suffisant d'entrées/sorties. Dans la suite du rapport, la carte Arduino Nano Every sera appelée "station air" et la carte Arduino Mega 2560 sera la "station sol", afin de les identifier et de distinguer leurs composants/modules respectifs.

La station sol est directement reliée à un ordinateur par liaison USB. Cela permet de l'alimenter et de récolter via le port série les données transmises par la station air. La station air quant à elle, est alimentée par un UBEC 5V 3A lui-même desservi par une batterie externe portable justement dimensionnée pour le système. (Voir la partie "[Énergie](#)".)

Vous trouverez en [annexe 1](#) leurs schémas de pinout respectifs [\[1, 2\]](#), qui nous ont été utiles pour déterminer les pins disponibles pour nos composants et capteurs, ainsi que d'identifier les pins correspondants aux communications UART, I2C et SPI.

### Unité de capture vidéo : ESP32-S3 IA CAM

L'unité de capture vidéo est située dans la station air, et est dédiée à l'acquisition et à l'enregistrement des images embarquées. Elle fonctionne indépendamment de l'unité de contrôle afin de ne pas perturber les performances de la télémétrie. Ce module dispose à la fois d'un microcontrôleur ESP32-S3 et d'une caméra, ce qui permet de capturer des images sans nécessiter de composants supplémentaires. De plus, un lecteur de carte microSD est disponible, permettant de stocker localement les données (figure 1).

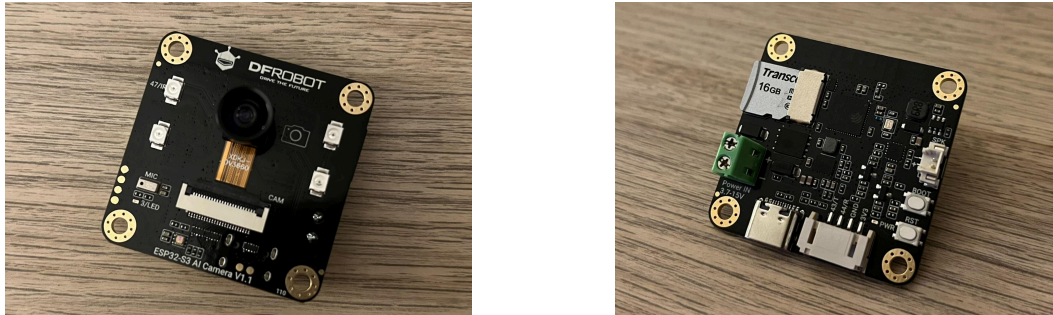
Cependant, ce module ne permet pas une transmission vidéo longue portée dans le cadre de ce projet, ce qui justifie le choix d'un enregistrement local plutôt qu'un retour vidéo en direct. Des images sont capturées, puis enregistrées en temps réel sur la carte microSD. Une fois le vol terminé, on peut récupérer la carte, et créer une vidéo à partir des images enregistrées.

Le déclenchement de la capture et de l'enregistrement se fait en branchant le module ESP32 au 5V délivré par l'UBEC. Il faut donc que le système soit au sol pour démarrer ou éteindre l'enregistrement. Une version ultérieure du projet pourrait permettre l'allumage/l'extinction du système à distance, grâce à un relais ou un MOSFET. (Voir la partie "[Améliorations envisageables](#)".)

Ainsi, dans ce projet, l'ESP32 assure la capture d'images embarquées, l'enregistrement de ces images sur la carte microSD, et un fonctionnement autonome une fois alimenté.







*Figure 1 : ESP32-S3 IA CAM DFRobot [3] (faces avant et arrière)*

## Capteur de température, d'humidité et de pression : BME280

Dans ce projet, le BME280 est utilisé afin d'acquérir la température, la pression atmosphérique et l'humidité. Le choix de ce capteur repose en particulier sur sa compacité et sa très faible masse, mais aussi sur sa capacité à regrouper plusieurs mesures au sein d'un seul composant et sa faible consommation énergétique (figure 2).

Grâce à la mesure de la pression atmosphérique, on peut déterminer l'altitude du modèle RC à l'aide d'une relation barométrique. Cette information vient compléter les données fournies par le module GPS. Cependant, cette mesure dépend des conditions météorologiques et n'est donc pas très précise, c'est plus une estimation qu'une mesure directe. En revanche une variation d'altitude (de part une variation de pression), peut être précisément mesurée grâce à ce module.



*Figure 2 : BME280*

Ainsi, dans ce projet, le capteur BME280 assure la mesure de la température, la mesure de l'humidité, la mesure de la pression atmosphérique ainsi qu'une estimation de l'altitude.

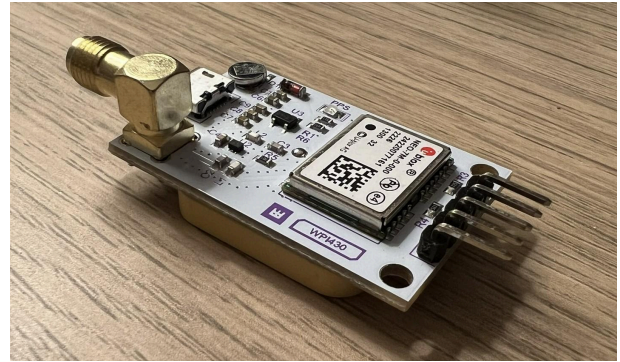
## Module GPS : U-blox NEO-7M

Le module GPS est utilisé afin de déterminer la position géographique du modèle en temps réel. Dans ce projet, un module U-blox NEO-7M (figure 3) est intégré à la station air pour fournir les coordonnées de latitude et de longitude uniquement : l'altitude étant plus précise avec le BME280 qu'avec ce module.

Le choix de ce module repose sur sa compatibilité avec les cartes Arduino, sa simplicité d'utilisation et son importante documentation. Il s'agit d'un module GPS couramment utilisé dans des projets embarqués, capable de fournir des données de position de manière autonome une fois le signal satellite acquis.



Lors de sa mise sous tension, un temps d'initialisation est nécessaire afin d'obtenir suffisamment de satellites pour des positions fiables, ce qui peut prendre plusieurs secondes, voire davantage selon les conditions de réception. De plus, la précision des mesures peut être affectée par l'environnement (obstacles, conditions météorologiques). On ne peut notamment pas utiliser ce GPS en intérieur à cause du signal satellite trop faible.

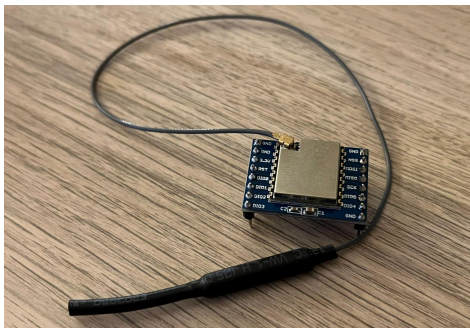


*Figure 3 : U-blox NEO-7M*

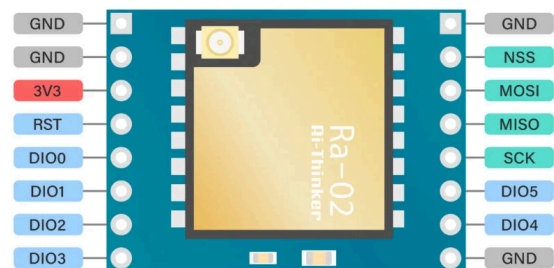
Ainsi, dans ce projet, le module GPS assure la mesure de la latitude et de la longitude.

## Communication longue portée : LoRa SX1278

La communication entre la station air et la station sol est assurée par une liaison longue portée basée sur la technologie LoRa (figure 4). Contrairement à d'autres technologies sans fil comme le Wi-Fi ou les modules NRF24L01, la modulation LoRa est spécifiquement conçue pour des applications à faible débit nécessitant une grande portée, ce qui correspond parfaitement aux contraintes de notre projet. Vous trouverez le pinout de ce module en figure 5.



*Figure 4 : Module LoRa SX1278*



*Figure 5 : Pinout LoRa SX1278 [4]*

Dans ce système, la station air envoie périodiquement les données issues des capteurs (température, pression, humidité, altitude et position GPS), tandis que la station sol reçoit ces informations et les transmet à un ordinateur. Cependant, cette technologie ne permet pas l'envoi de données volumineuses comme un flux vidéo. Ceci impose donc une séparation entre la télémétrie et la vidéo, enregistrée localement sur l'ESP32.

L'utilisation de la bande de fréquence 433 MHz permet d'éviter les interférences avec la radiocommande de l'avion, opérant en 2,4 GHz, tout en respectant les bandes de fréquence autorisées en Europe. (Voir la partie "[Sécurité et contraintes](#)".)



## Énergie

### Dimensionnement énergétique

Le système embarqué est alimenté par une batterie externe Li-ion 7.4V, associée à un module UBEC délivrant une tension régulée de 5V. Ce choix permet d'assurer une alimentation stable pour l'ensemble des composants. On dispose aussi d'un convertisseur de tension DC-DC 5V-3.3V pour les composants qui ont besoin d'une plus basse tension.

Afin de vérifier le bon dimensionnement de l'alimentation, nous avons réalisé une estimation des courants consommés par chaque composant :

| Composant          | Courant maximum (pics de courant)     |
|--------------------|---------------------------------------|
| Arduino Nano Every | 50 mA <a href="#">[5]</a>             |
| ESP32-S3 IA CAM    | 200 mA <a href="#">[6]</a>            |
| U-blox NEO-7M      | 50 mA <a href="#">[7]</a>             |
| LoRa SX1278        | 120 mA (émission) <a href="#">[8]</a> |
| BME 280            | 5 mA <a href="#">[9]</a>              |
| <b>Total :</b>     | <b>425 mA</b>                         |

Avec une marge de sécurité, on estime alors un courant total maximum de 500 mA. Cette valeur reste bien inférieure à la capacité maximale de l'UBEC de 3 A, garantissant un fonctionnement fiable du système.

### Autonomie

L'autonomie du système dépend de la capacité de la batterie utilisée et du courant consommé. Ici, nous utilisons une batterie Li-ion de capacité 3000 mAh. On peut alors estimer l'autonomie :

$$Autonomie(h) = \frac{Capacité(mAh)}{Courant(mA)} = \frac{3000}{500}(h) = 6h$$

En pratique, il est nécessaire de prendre en compte la décharge partielle de la batterie, les pertes énergétiques et son vieillissement. L'autonomie est ainsi estimée à environ 5h, valeur largement supérieure à la durée de vol d'un avion RC (2 à 30 minutes).



## Station sol

### Retour des coordonnées GPS

La station sol reçoit via LoRa les données transmises par la station air. Ces informations sont décodées par l'Arduino puis envoyées vers un ordinateur via une liaison série USB, où un programme Python les lit en temps réel. Les coordonnées de latitude et de longitude sont transmises sous forme de texte. Le script Python les lit ligne par ligne, puis utilise des expressions régulières pour détecter et extraire les valeurs, qui sont ensuite stockées afin de reconstituer le trajet du modèle.

La visualisation est réalisée à l'aide de la bibliothèque "Folium", qui permet de générer une carte basée sur des données cartographiques (OpenStreetMap). Le résultat est exporté sous forme d'un fichier HTML, ouvrable dans un navigateur web. À chaque nouvelle position reçue : un point est ajouté sur la carte ; le trajet est tracé sous forme de lignes reliant les différentes positions.

Pour visualiser cette représentation en temps réel et les différentes positions du modèle RC, il suffit d'actualiser la page. Il faut cependant noter que la précision dépend des coordonnées envoyées par le module GPS, et que celles-ci peuvent varier de manière incohérente selon l'environnement et les satellites disponibles.

Ci-dessous figure 6, un exemple de visualisation que l'on obtient. Le point rouge correspond aux premières coordonnées reçues ; le point vert, les dernières. On peut voir que le chemin parcouru est retracé.

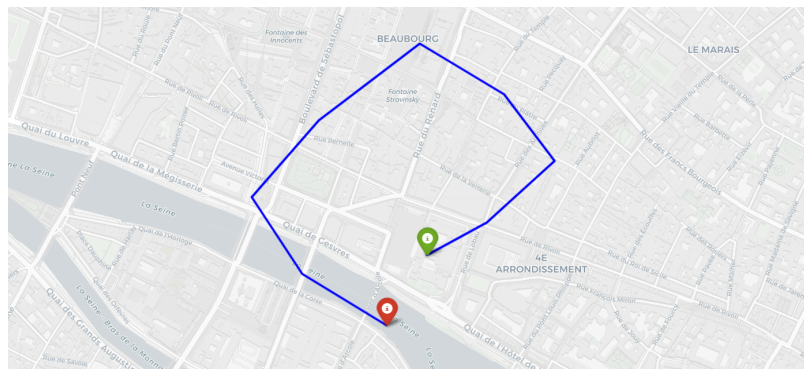


Figure 6 : Exemple de visualisation obtenue

### Retour des informations télémétriques

Les informations télémétriques sont récupérées de la même manière que les coordonnées GPS : une expression régulière détecte le mot-clé correspondant et extrait le nombre situé en fin de ligne. Ce nombre est alors stocké dans une variable dédiée. Lorsque les cinq variables (température, humidité, pression, altitude, RSSI) sont toutes remplies, les valeurs sont écrites dans un fichier CSV (figure 7).





À chaque nouveau paquet enregistré, le script relit l'intégralité du fichier CSV et génère quatre graphiques qui sont affichés simultanément dans une fenêtre Spyder : température, humidité, RSSI, et un graphique combinant pression/altitude. Les axes sont ajustés automatiquement autour des valeurs réelles pour éviter une lecture trompeuse des variations (figure 8).

|    | A           | B        | C        | D        | E     |
|----|-------------|----------|----------|----------|-------|
| 1  | temperature | humidite | pression | altitude | rsi   |
| 2  | 14.18       | 50.33    | 1008.58  | 86.65    | -89.0 |
| 3  | 14.18       | 50.53    | 1008.55  | 86.64    | -89.0 |
| 4  | 14.19       | 50.18    | 1008.57  | 86.74    | -88.0 |
| 5  | 14.21       | 50.08    | 1008.57  | 86.71    | -89.0 |
| 6  | 14.39       | 50.04    | 1008.6   | 86.36    | -92.0 |
| 7  | 14.34       | 50.07    | 1008.6   | 86.45    | -92.0 |
| 8  | 14.29       | 50.2     | 1008.58  | 86.73    | -92.0 |
| 9  | 14.3        | 49.75    | 1008.57  | 86.88    | -89.0 |
| 10 | 14.3        | 49.71    | 1008.58  | 86.54    | -89.0 |
| 11 | 14.28       | 49.58    | 1008.58  | 86.92    | -90.0 |
| 12 | 14.29       | 49.72    | 1008.57  | 87.0     | -89.0 |
| 13 | 14.25       | 49.5     | 1008.57  | 86.77    | -93.0 |
| 14 | 14.28       | 49.49    | 1008.59  | 86.56    | -92.0 |
| 15 | 14.29       | 49.48    | 1008.58  | 86.65    | -93.0 |
| 16 | 14.31       | 49.72    | 1008.61  | 86.33    | -93.0 |
| 17 | 14.31       | 49.46    | 1008.55  | 86.74    | -93.0 |
| 18 | 14.35       | 49.74    | 1008.58  | 86.64    | -91.0 |
| 19 | 14.33       | 50.05    | 1008.58  | 86.73    | -96.0 |
| 20 | 14.32       | 50.41    | 1008.58  | 86.65    | -92.0 |
| 21 | 14.3        | 50.43    | 1008.59  | 86.53    | -94.0 |
| 22 | 14.3        | 50.35    | 1008.58  | 86.29    | -93.0 |
| 23 | 14.31       | 50.95    | 1008.59  | 86.48    | -94.0 |

Figure 7 : Fichier CSV obtenu

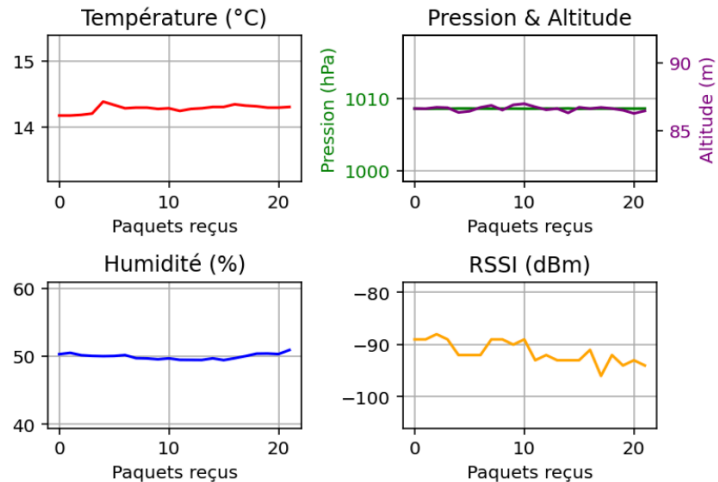


Figure 8 : Graphiques obtenus

## Exploitation vidéo post-vol

À l'issue du vol, la carte microSD est extraite du module afin de récupérer les images enregistrées. Celles-ci sont sauvegardées sous forme de fichiers individuels (*img\_0.jpg*, *img\_1.jpg*, *img\_2.jpg*, ...), correspondant à une acquisition au cours du temps. Afin de reconstruire une vidéo à partir de ces images, un outil de traitement vidéo est utilisé. Dans ce projet, le logiciel FFmpeg est employé. Après avoir ouvert un terminal dans le dossier contenant les images, la commande suivante est exécutée :

```
ffmpeg -framerate 10 -i img_%d.jpg -c:v libx264 -pix_fmt yuv420p video.mp4
```

Cette commande permet d'assembler les images en une séquence vidéo au format mp4, avec une fréquence de 10 images par seconde. Le fichier obtenu peut ensuite être lu sur un ordinateur afin de visualiser le vol.





## Câblage et Programmation

Chaque module utilise un protocole de communication différent afin d'optimiser les performances. Le nombre limité de broches de l'Arduino Nano impose de réduire les connexions pour garantir un système fiable. De plus, selon le microcontrôleur utilisé, les modules/composants ne se branchent pas de la même manière, et il peut même être nécessaire de spécifier au programme les emplacements choisis suivant la carte utilisée. C'est le cas notamment pour le module air, lors de l'association Arduino Nano Every et module LoRa SX1278.

Vous trouverez en [annexe 2](#) des photographies de la première version du système. Ce prototype fonctionnel est câblé sur une breadboard, ce qui facilite les phases de développement et de test. Dans une version finale, la réalisation d'un circuit imprimé (PCB) permettrait de réduire l'encombrement et la masse de la station air en particulier (actuellement de 234g), tout en améliorant la fiabilité du câblage et la qualité de l'intégration.

Les modules LoRa communiquent avec Arduino via le protocole SPI [\[10\]](#). Voici ci-dessous le pinout associé.

| LoRa SX1278 | Arduino Nano Every<br>(module air) | Arduino Mega 2560<br>(module sol) |
|-------------|------------------------------------|-----------------------------------|
| 3.3V        | 3.3V                               | 3.3V                              |
| GND         | GND                                | GND                               |
| RST         | D9                                 | D9                                |
| DIO0        | D2                                 | D2                                |
| NSS         | D8                                 | D53                               |
| MOSI        | D11                                | D51                               |
| MISO        | D12                                | D50                               |
| SCK         | D13                                | D52                               |

Le module GPS U-blox NEO-7M communique avec l'Arduino via le protocole UART [\[11\]](#) et le module BME280 communique via le protocole I<sup>2</sup>C [\[12\]](#). Voici ci-dessous les pinouts associés :

| BME280 | U-blox NEO-7M | Arduino Nano Every<br>(module air) |
|--------|---------------|------------------------------------|
|        | VCC           | 5V                                 |
| VCC    |               | 3.3V                               |
| GND    | GND           | GND                                |
|        | TXD           | D4 (RX)                            |
|        | RXD           | D3 (TX)                            |
| SCL    |               | A5                                 |
| SDA    |               | A4                                 |



## LED d'état

Ont été ajoutées 3 LED d'état dans la station air. La verte s'allume une fois que tous les composants ont démarré correctement, et clignote une fois pour chaque composant correctement initialisé. Celle-ci est branchée sur le pin D5. La blanche ne s'allume que lors de l'initialisation des composants afin de signifier le bon démarrage du système, elle est branchée sur le pin D7. Enfin, la rouge ne s'allume que lorsque l'initialisation de l'un des composants a échoué, elle est branchée sur le pin D6. Toutes les LED sont en série avec une résistance de  $330\Omega$ .

## Distribution de l'énergie

Comme vu précédemment, on dispose d'un UBEC 5V 3A pour réguler la tension. Celui-ci, positionné en parallèle de la batterie, fournit l'énergie à l'ESP32 et à un convertisseur de tension DC-DC 5V-3.3V. On utilise ici une breadboard, dont un rail fournit le 5V de l'UBEC et l'autre rail le 3.3V du convertisseur de tension. On peut alors brancher le Vin de l'Arduino Nano sur le rail 5V, chaque composant sur le rail correspondant, et connecter les GND ensembles.

Vous trouverez en [annexe 3](#) le schéma de câblage de la station air, réalisé avec KiCad. À noter que ce schéma privilégie une représentation proche du câblage réel afin de faciliter la compréhension du montage. Il permet une visualisation d'ensemble propre et complète.

## Programmation

L'ensemble des codes sources développés dans le cadre de ce projet est disponible à la consultation et la réutilisation sur GitHub, dont le lien est fourni en [annexe 4](#).

De plus, en [annexe 5](#) sont disponibles les logigrammes de chacun de ces codes, afin de suivre la logique de programmation.



## Informations complémentaires

### Sécurité et contraintes

#### Fréquences

L'utilisation de modules LoRa impose le respect des réglementations en vigueur concernant les bandes de fréquence radio. En Europe, ces dispositifs peuvent opérer dans plusieurs bandes ISM, notamment autour de 433 MHz et 868 MHz, sous certaines conditions [\[13\]](#).

Dans le cadre de ce projet, ces contraintes sont respectées en limitant la fréquence d'émission des données télémétriques et en utilisant des puissances adaptées. De plus, le choix de la bande 433 MHz permet d'éviter les interférences avec la radiocommande de l'avion, fonctionnant en 2,4 GHz.

#### Communication LoRa coupée

Dans le cas où la communication entre les deux modules LoRa serait coupée (à cause d'une batterie vide, ou d'un reset de la carte Arduino par exemple), la station sol ne reçoit plus les données télémétriques. Mais cela n'impacte en rien le bon fonctionnement de l'avion RC, et donc la sécurité des personnes alentour.

### Améliorations envisageables

Plusieurs améliorations techniques peuvent être envisagées afin d'augmenter les performances, l'ergonomie et l'autonomie du système.

Tout d'abord, une évolution importante consisterait à permettre le contrôle à distance de l'unité de capture vidéo. Actuellement, l'ESP32 est alimenté manuellement avant le vol. L'intégration d'un dispositif de commutation électronique, tel qu'un MOSFET ou un relais piloté par l'Arduino, permettrait d'allumer ou d'éteindre la caméra à distance via la liaison LoRa. Cela offrirait une meilleure gestion de l'énergie en évitant de filmer en continu, et permettrait d'adapter l'enregistrement aux phases de vol pertinentes.

Ensuite, la qualité des images capturées pourrait être améliorée par l'ajout d'un système de stabilisation. Il serait intéressant d'intégrer un capteur inertiel (gyroscope par exemple) qui permettrait de mesurer les mouvements de l'appareil et, à terme, de compenser les vibrations soit mécaniquement (via un support stabilisé), soit numériquement en post-traitement. Ce capteur pourrait également être exploité pour d'autres fonctions, comme l'analyse du comportement en vol ou l'estimation de l'inclinaison du modèle RC.



## Conclusion

Ce projet, bien que réalisé à une échelle expérimentale, met en évidence l'intérêt des systèmes embarqués pour la collecte et la transmission de données environnementales. L'utilisation de technologies sobres en énergie et adaptées aux longues distances illustre une approche compatible avec les enjeux du développement durable.

À terme, ce type de système pourrait servir à des applications concrètes telles que la surveillance environnementale ou l'analyse de conditions atmosphériques locales, contribuant ainsi à une meilleure compréhension de notre environnement par une méthode plus neutre pour la planète.



*Figure 9 : Intégration de la station téléométrique sur le modèle RC*



## Bibliographie

1. Arduino. *Arduino Nano Every Pinout diagram*. Document technique.  
Disponible sur : <https://docs.arduino.cc/resources/pinouts/ABX00028-full-pinout.pdf>.  
Consulté en mars 2026.
2. Arduino. *Arduino Mega 2560 Pinout diagram*. Document technique.  
Disponible sur : <https://docs.arduino.cc/resources/pinouts/A000067-full-pinout.pdf>.  
Consulté en mars 2026.
3. DFRobot. *Gravity: ESP32-S3 AI Camera Module – Documentation technique*.  
Disponible sur : <https://wiki.dfrobot.com/dfr1154/>.  
Consulté en mars 2026.
4. ADIY. *LoRa Module RA-02 SX1278 Module*. Image produit.  
Disponible sur : <https://adiy.in/shop/lora-module-ra-02-sx1278-module/>.  
Consulté en avril 2026.
5. Arduino. *Arduino Nano Every – Technical Specifications*.  
Disponible sur : <https://docs.arduino.cc/hardware/nano/>.  
Consulté en avril 2026.
6. Espressif Systems. *ESP32-S3 – Datasheet*.  
Disponible sur : [https://documentation.espressif.com/esp32-s3\\_datasheet\\_en.pdf](https://documentation.espressif.com/esp32-s3_datasheet_en.pdf).  
Consulté en avril 2026.
7. u-blox. *NEO-7M – Data Sheet*.  
Disponible sur :  
[https://content.u-blox.com/sites/default/files/products/documents/NEO-7\\_DataSheet\\_%28UBX-13003830%29.pdf](https://content.u-blox.com/sites/default/files/products/documents/NEO-7_DataSheet_%28UBX-13003830%29.pdf).  
Consulté en avril 2026.
8. Semtech. *SX1278 – LoRa Transceiver Datasheet*.  
Disponible sur : <https://www.semtech.com/products/wireless-rf/lora-connect/sx1278>.  
Consulté en avril 2026.
9. Bosch Sensortec. *BME280 – Environmental Sensor Datasheet*.  
Disponible sur :  
<https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>.  
Consulté en avril 2026.
10. Circuit Basics. *Basics of the SPI Communication Protocol*.  
Disponible sur : <https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>.  
Consulté en mai 2026.
11. Arduino Documentation. *Unlocking the Power of UART: A Guide to Asynchronous Communication*.  
Disponible sur : <https://docs.arduino.cc/learn/communication/uart/>.  
Consulté en mai 2026.
12. Circuit Basics. *Basics of the I2C Communication Protocol*.  
Disponible sur : <https://www.circuitbasics.com/basics-of-the-i2c-communication-protocol/>.  
Consulté en mai 2026.
13. AFNOR. *ETSI NF EN 300220 – équipements radio courte portée*.  
Norme européenne.  
Consulté en mars 2026.

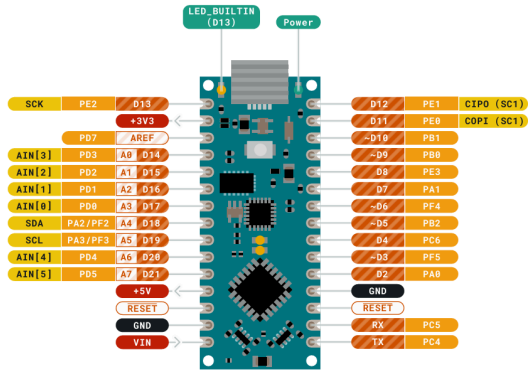




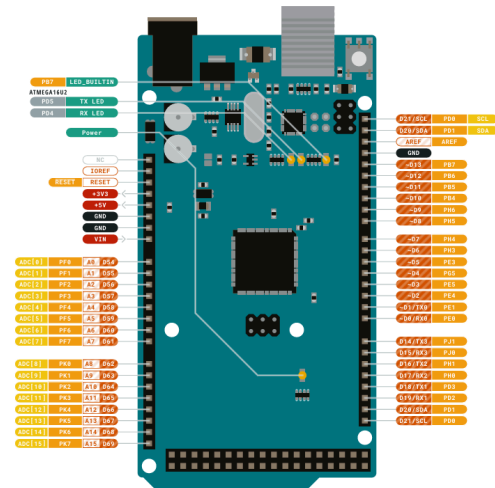
## Annexes

### Annexe 1 : Pinout des cartes Arduino utilisées

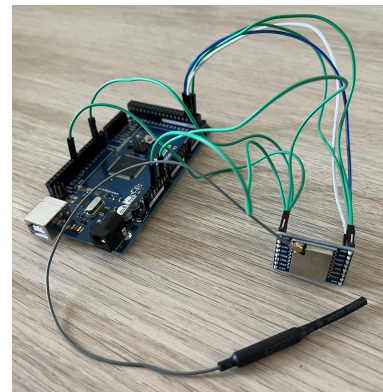
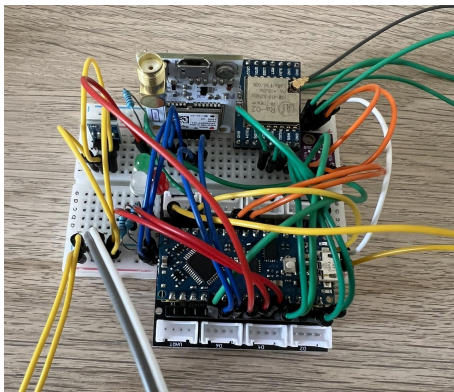
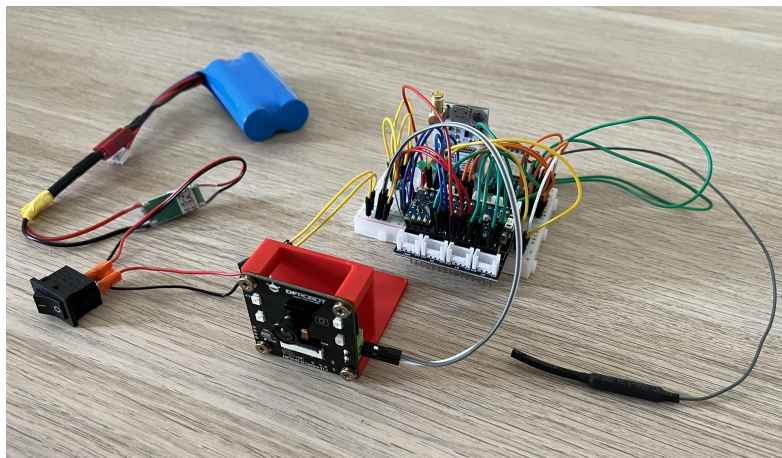
#### Pinout Arduino Nano Every [\[1\]](#)

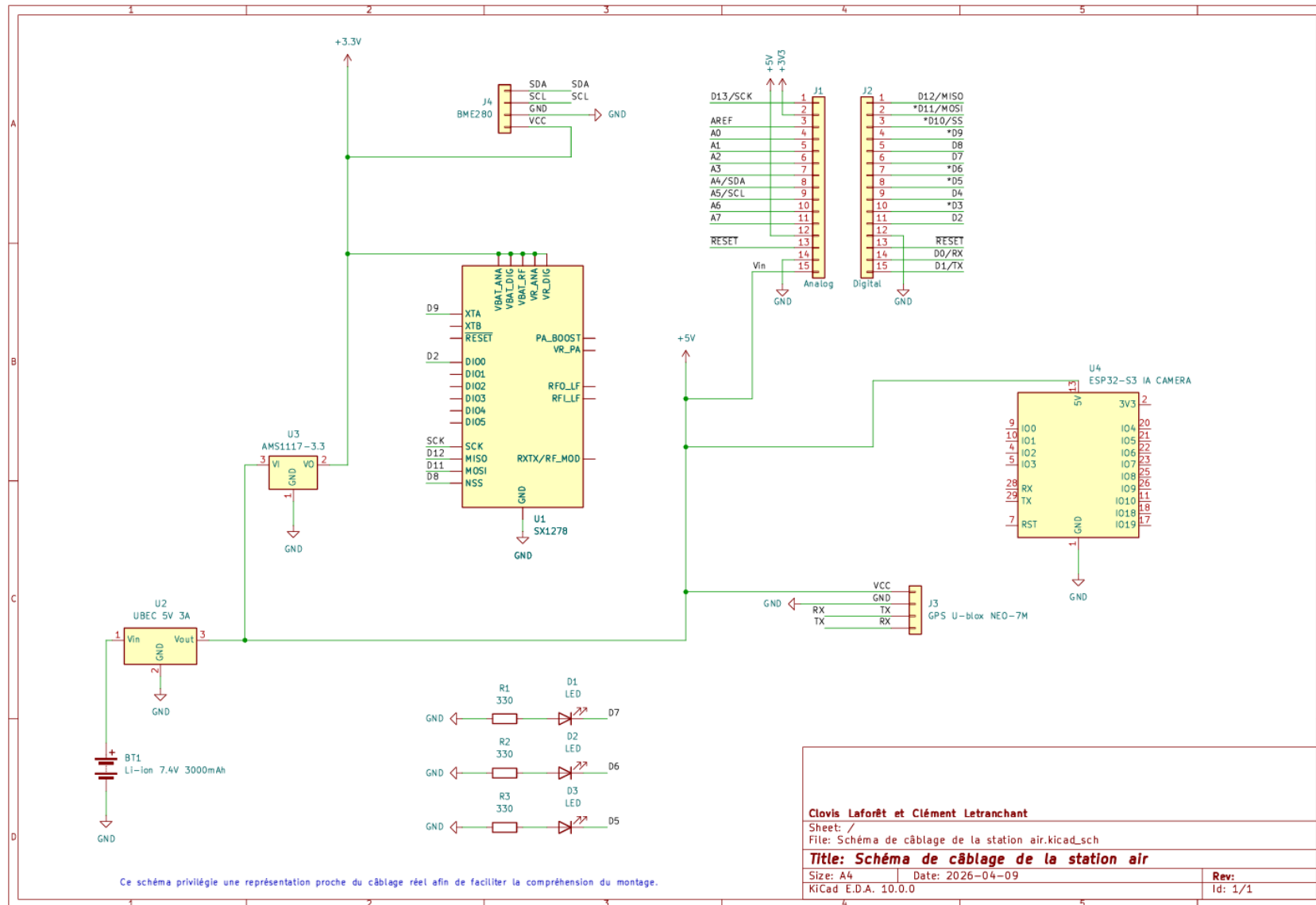


#### Pinout Arduino Mega 2560 [\[2\]](#)



### Annexe 2 : Photos de la première version du système (station air et station sol)



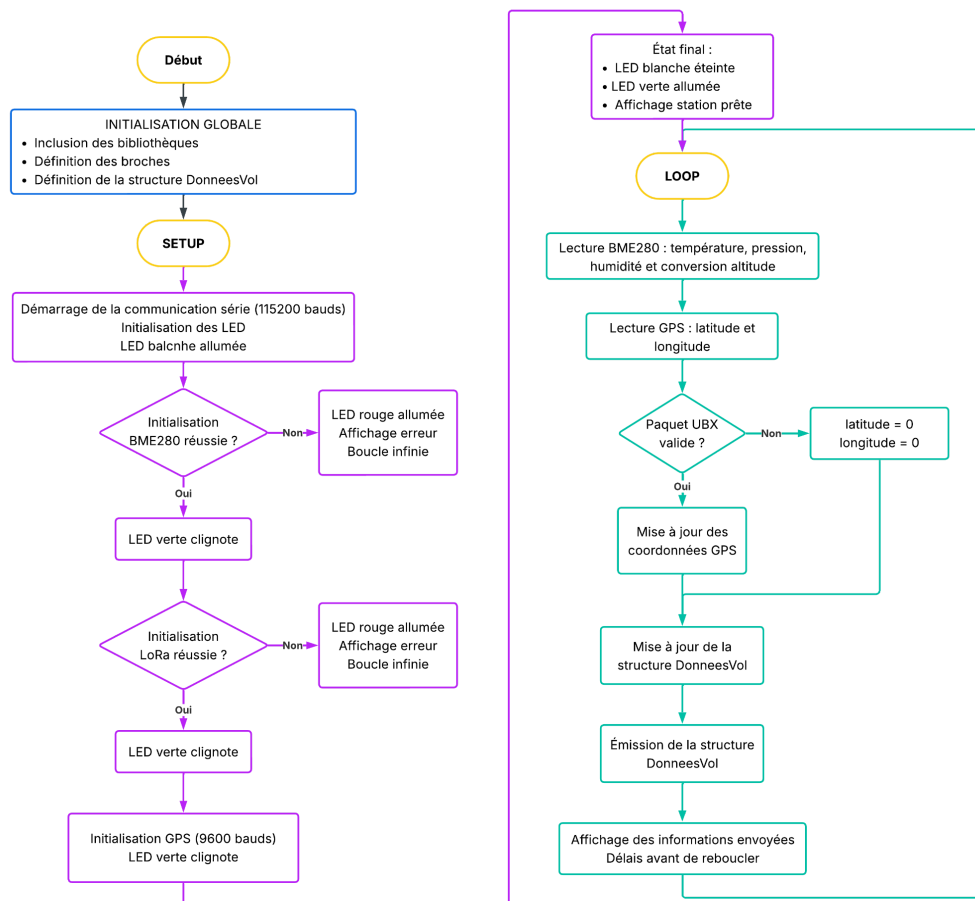
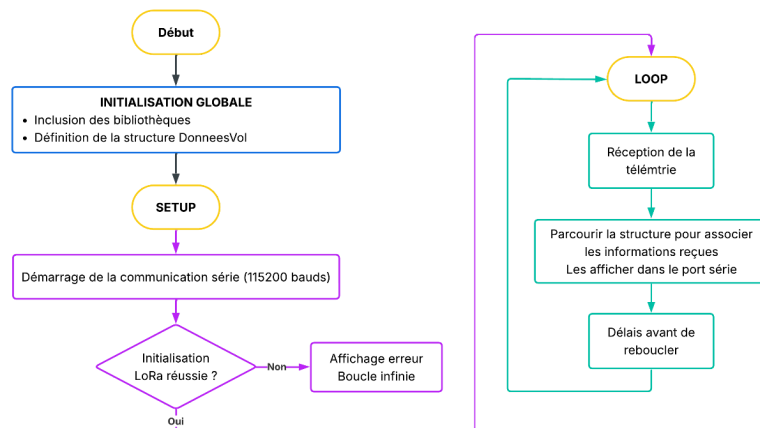


Annexe 3 : Schéma de câblage de la station air

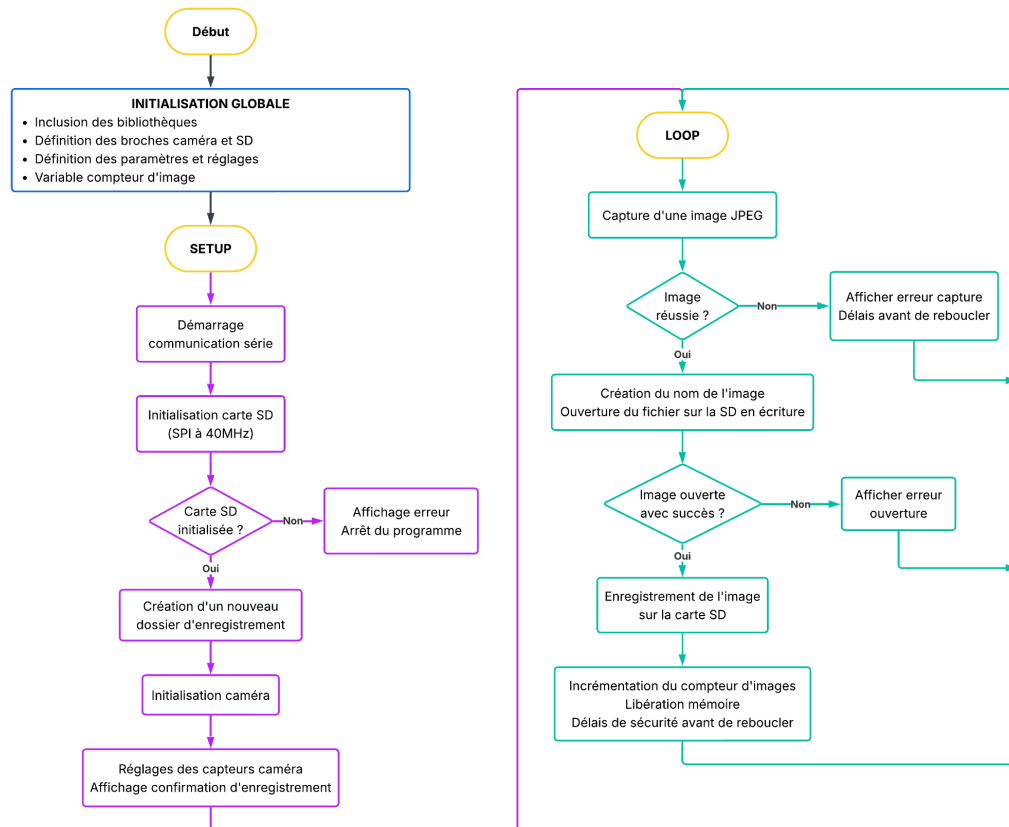


Annexe 4 : Accès aux codes sources

Clovis Laforêt. *Station-telemetry*. Dépôt GitHub.  
Disponible sur <https://github.com/clovis-laforet/Station-télémetrique>.

Annexe 5 : Logigrammes**Logigramme de la station air****Logigramme de la station sol**

## Logigramme de l'ESP32-S3 IA CAM



## Logigramme de l'affichage des données télémétriques

